

❖ Construcción de una API RESTful con Express.js y métodos HTTP

Construir una API RESTful con Express.js implica configurar un servidor Node.js, definir rutas y usar métodos HTTP (GET, POST, PUT, DELETE) para gestionar recursos CRUD. Se instalan dependencias (npm install express), se estructuran carpetas (routes, controllers) y se utiliza app.use(express.json()) para manejar datos en formato JSON.

Pasos Clave para la Construcción:

- **Inicialización:** Ejecutar npm init -y para crear el package.json e instalar Express con npm install express.
- **Servidor Básico:** Crear un archivo index.js o app.js e importar Express para configurar el servidor.
- **Estructura RESTful:** Organizar el código en carpetas como routes (rutas), controllers (lógica) y models (datos).
- **Métodos HTTP (CRUD):**
 - **GET:** Obtener recursos (app.get()).
 - **POST:** Crear nuevos recursos (app.post()).
 - **PUT/PATCH:** Actualizar recursos existentes (app.put()).
 - **DELETE:** Eliminar recursos (app.delete()).

Dejo la extensión al plugin:

https://chromewebstore.google.com/detail/json-viewer-pro/eiffpmocdbdmebjapokkhhbfmdgjcc?hl=es&utm_source=ext_sidebar

❖ Lectura y Escritura de Archivos en Node.js con FS y Path

La lectura y escritura de archivos en Node.js se gestiona principalmente con el módulo nativo fs (File System), utilizando path para manejar rutas de forma segura. Se recomienda usar métodos asíncronos (fs.readFile, fs.writeFile) con promesas (fs/promises) o callbacks para no bloquear el hilo principal, empleando UTF-8 para texto.

```
javascript
const fs = require('fs').promises; // Usando promesas
const path = require('path');

async function gestionarArchivo() {
  const ruta = path.join(__dirname, 'ejemplo.txt');

  // Escritura
  await fs.writeFile(ruta, 'Hola desde Node.js', 'utf-8');
  console.log('Archivo creado');

  // Lectura
  const contenido = await fs.readFile(ruta, 'utf-8');
  console.log('Contenido:', contenido);
}
```

}

Principales Métodos del Módulo FS

- **Lectura:**
 - `fs.readFile(path, options, callback)`: Asíncrono, lee todo el archivo.
 - `fs.readFileSync(path, options)`: Síncrono, bloquea la ejecución.
- **Escritura:**
 - `fs.writeFile(path, data, options, callback)`: Asíncrono, crea o sobrescribe.
 - `fs.writeFileSync(path, data, options)`: Síncrono, sobrescribe.
- **Uso de Path:**
 - `path.join(__dirname, 'archivo.txt')`: Asegura la ruta correcta independientemente del sistema operativo.

Mejores Prácticas

- **Asincronía:** Prefiere siempre `fs.promises` o `callbacks` para evitar bloqueos en el servidor.
- **Codificación:** Especifica siempre `'utf-8'` al leer o escribir archivos de texto para obtener strings en lugar de Buffers.
- **Manejo de Errores:** Usa bloques `try-catch` al trabajar con `async/await` para manejar archivos inexistentes o falta de permisos.
- **Streams:** Para archivos muy grandes, utiliza `fs.createReadStream` y `fs.createWriteStream` para optimizar el uso de memoria.

❖ Creación de Usuarios con Método POST en Node.js

Para crear usuarios con el método POST en Node.js, utiliza Express para definir una ruta `/users` que reciba datos JSON en `req.body`, válidalos y guárdalos (ej. MongoDB o SQL). Es esencial usar `express.json()` para procesar el cuerpo de la solicitud y responder con el estado 201.

Pasos clave para la creación de usuarios:

- **Configuración:** Requiere `express` y un middleware de análisis de cuerpo como `express.json()` o `body-parser` para interpretar los datos recibidos.
- **Ruta POST:** Define `app.post('/users', (req, res) => { ... })` para manejar la solicitud.
- **Procesamiento:** Obtén los datos del usuario desde `req.body`.
- **Persistencia:** Guarda el usuario en la base de datos y genera un ID único.
- **Respuesta:** Envía una respuesta HTTP 201 (Created) con los datos creados.

javascript

```
const express = require('express');  
const app = express();
```

```
// Middleware para entender JSON  
app.use(express.json());
```

```
app.post('/users', (req, res) => {
```

```
const { nombre, email } = req.body;

// Aquí iría la lógica de base de datos
const nuevoUsuario = { id: Date.now(), nombre, email };

// Respuesta exitosa
res.status(201).json(nuevoUsuario);
});

app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

❖ Actualización de Usuarios con Node.js y Express

Actualizar usuarios con Node.js y Express implica crear una ruta PUT o PATCH que reciba datos, valide la información y actualice la base de datos. Se utilizan middlewares para manejar la petición y respuestas JSON para confirmar el éxito, comúnmente integrando validación de datos para seguridad.

Pasos Clave para la Actualización de Usuarios:

- **Configuración de la Ruta (app.put/app.patch):** Definir una ruta, por ejemplo, /users/:id, para identificar qué usuario actualizar.
- **Gestión de Peticiones (req.body):** Capturar los nuevos datos enviados por el usuario en el cuerpo de la petición (JSON).
- **Actualización en Base de Datos:** Utilizar un ORM (como Prisma o Mongoose) o consultas SQL para modificar el registro específico.
- **Validación de Datos:** Antes de actualizar, verificar que la información cumpla con los requisitos (ej. formato de email) para evitar errores.
- **Respuesta de la API:** Enviar una respuesta JSON confirmando la actualización o indicando errores.

Mejores Prácticas:

- **Autenticación y Autorización:** Asegurarse de que solo usuarios autenticados y autorizados puedan actualizar datos, a menudo usando Passport.js o JWT.
- **Validación:** Implementar validaciones en los datos recibidos (ej. usando Joi o Express-validator) para mantener la integridad de la base de datos.
- **Arquitectura Limpia:** Separar la lógica de negocio de las rutas de Express para mejorar la mantenibilidad del código.

❖ Validaciones en POST y PUT para Datos de Usuario en APIs

Las validaciones de usuario en APIs REST aseguran la integridad de los datos. **POST** crea recursos nuevos (requiere validación estricta de campos obligatorios como email/contraseña), mientras que **PUT** actualiza o reemplaza recursos existentes, validando que el ID exista y los datos sean válidos. Ambos requieren verificaciones de formato, longitud y seguridad.

Validaciones Comunes (POST y PUT)

- **Obligatoriedad:** Verificar campos requeridos (nombre, email).
- **Tipos de Datos:** Asegurar que los formatos sean correctos (ej. email válido, números, strings).
- **Longitud:** Limitar longitud de campos (ej. contraseñas mínimo 8 caracteres).
- **Autenticación:** Proteger endpoints con tokens (Bearer token) para asegurar que el usuario tenga permisos.

Diferencias Clave en Validaciones

- **POST (/users):**
 - Verificar que el usuario **no exista** previamente (ej. correo duplicado).
 - Validar que el cuerpo de la solicitud no esté vacío.
- **PUT (/users/{id}):**
 - Verificar que el **ID existe** en la base de datos antes de intentar actualizar.
 - Validar que el ID en la URL coincida con el ID en el cuerpo de la solicitud, si aplica

❖ Eliminar Usuarios en un CRUD usando Node.js y Express

Eliminar un usuario en un CRUD con Node.js y Express implica crear una ruta HTTP DELETE o GET (si se usan formularios HTML), capturar el ID del usuario desde req.params, y ejecutar una consulta DELETE FROM (SQL) o deleteOne() (MongoDB) en la base de datos, manejando errores con try-catch para confirmar la acción. [1, 2, 3]

Pasos para eliminar usuarios (CRUD)

1. **Definir la Ruta (Routes):** Se utiliza app.delete('/users/:id', ...) para APIs RESTful o una ruta get para eliminar vía enlace.
2. **Controlador (Controller):** Captura el ID con const { id } = req.params;
3. **Consulta a Base de Datos:**
 - **MySQL:** db.query("DELETE FROM users WHERE id = ?", [id], ...).
 - **MongoDB/Mongoose:** User.findByIdAndDelete(id).
4. **Respuesta:** Enviar una confirmación JSON o redirigir a la lista de usuarios.

javascript

```
// Ejemplo utilizando Express y MySQL
app.delete('/user/:id', (req, res) => {
  const { id } = req.params;
  pool.query('DELETE FROM users WHERE id = ?', [id], (error, results) => {
    if (error) {
      return res.status(500).json({ error: 'Error al eliminar' });
    }
  });
});
```

```
    }  
    res.json({ message: 'Usuario eliminado correctamente' });  
  });  
});
```

Recomendaciones

- **Confirmación:** Asegúrate de implementar una confirmación en el frontend antes de realizar la petición de eliminación.
- **Seguridad:** Utiliza parámetros preparados para evitar inyecciones SQL.
- **Eliminación Lógica vs Física:** Considera desactivar usuarios (actualizar campo activo a false) en lugar de borrarlos permanentemente de la base de datos (eliminación física).